

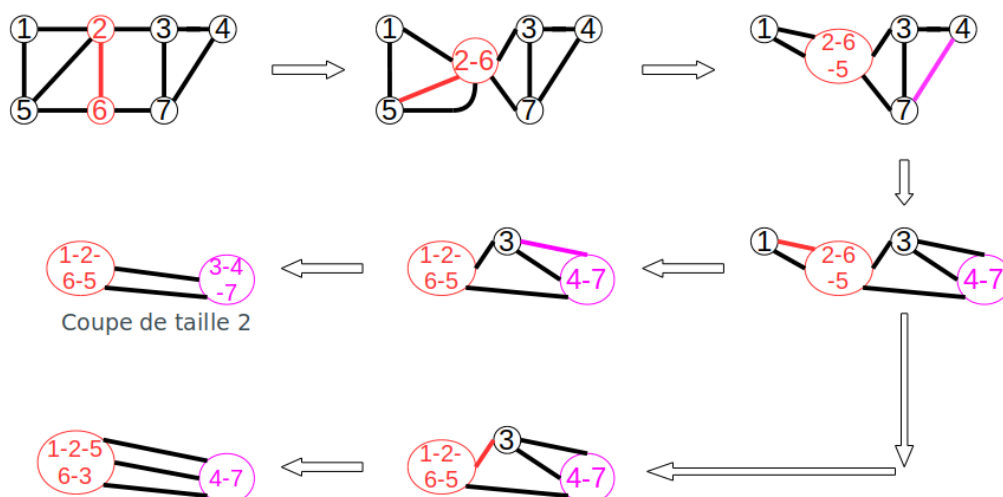
TP - algorithmes probabilistes ; durée : 3h

Vous déposerez une archive contenant le code source du TP.

1 Algorithme de Karger pour la coupe minimale

L'algorithme de Karger est un algorithme probabiliste de type Monte-Carlo qui retourne une coupe dont on espère qu'elle est minimale. Cet algorithme, vu en cours, est basé sur le principe de contraction d'une arête. A chaque étape, l'algorithme contracte une arête au hasard et fusionne ainsi les deux extrémités. Ainsi, le nombre de sommets décroît de 1 à chaque étape. Lorsqu'il ne reste que deux sommets, on obtient une coupe dont on espère quelle est minimale et la taille de la coupe est le nombre d'arêtes entre les deux sommets restants.

L'illustration ci-dessous, issue du cours, montre deux exécutions possibles de l'algorithme de Karger qui conduisent à une coupe minimale de taille 2 et à une coupe non minimale de taille 3. A chaque fois, l'arête contractée est mise en couleur.



Après chaque étape, les arêtes entre deux sommets peuvent être multiples. On parle alors de multigraphes. Dans notre cas, les multigraphes sont non orientés (les arêtes sont bidirectionnelles) et sans boucle (pas d'arête partant d'un sommet et arrivant au même sommet). Pour représenter un multigraphe non orienté (avec ou sans boucle), nous utiliserons la classe `Multigraph` fournie avec le TP. Dans cette classe, chaque nœud est un ensemble d'entiers, et plusieurs arêtes peuvent exister entre la même paire de nœuds. La classe `Multigraph` contient trois attributs :

- `graph` est un dictionnaire où chaque clé est un frozenset représentant un nœud, et chaque valeur est un autre dictionnaire où les clés sont des frozensets voisins et les valeurs sont des entiers représentant le nombre d'arêtes entre les nœuds.
- `m` est le nombre total d'arêtes dans le multigraphe.
- `n` est le nombre total de nœuds uniques dans le multigraphe.

Un exemple construisant le cinquième multigraphe de l'exécution ci-dessus est donnée dans le fichier fourni.

Question 1 : Ecrire une fonction `random_multi(m,n)` qui retourne un multigraphe aléatoire à n sommets (numérotés de 1 à n) et m arêtes. Pour cela, il suffit de répéter l'évènement suivant à m reprises : choisir une arête au hasard et l'ajouter au multigraphe (une arête peut être choisie à plusieurs reprises puisque nous considérons des multigraphes).

Ecrire également une fonction `illustration_multi()` qui retourne le premier multigraphe de l'illustration ci-dessus.

Remarque : ces fonctions pourront être utilisées pour tester vos fonctions.

Question 2 : Ecrire une fonction `contraction(multi,u,v)` qui prend en entrée un multigraphe `multi` et qui contracte l'arête entre les sommets étiquetés par `u` et `v`. Le multigraphe sera directement modifié (inutile d'en faire une copie). Le nouveau nœud obtenu sera l'union des deux nœuds contractés.

Question 3 : Ecrire une fonction `random_arete(multi)` qui prend en entrée un multigraphe `multi` à n sommets et qui retourne une arête aléatoire (u, v) , choisie **uniformément** parmi toutes les arêtes. La probabilité de l'arête (u, v) est donc proportionnelle au nombre d'arêtes entre u et v .

Question 4 : Ecrire une fonction `karger(multi)` qui applique l'algorithme de Karger sur le multigraphe `multi`. L'algorithme retourne un multigraphe à deux sommets représentant une coupe du multigraphe `multi`. La taille de la coupe est le nombre d'arêtes entre ces deux sommets.

Remarque 1 : si le multigraphe en entrée contient plus de 3 composantes connexes, l'algorithme de Karger se termine avant qu'il ne reste deux sommets. Dans ce cas, le multigraphe obtenu à la fin contient autant de sommets que de composantes connexes et aucune arête. La taille de la coupe minimale est 0.

Remarque 2 : ne dupliquez pas le multigraphe `multi`. Travaillez directement avec.

Question 5 : En cours, nous avons étudié la probabilité que l'algorithme de Karger retourne une coupe de taille minimale. A partir du cours, implémentez une fonction `karger_certitude(multi,certitude)` qui retourne un multigraphe à deux sommets représentant une coupe du multigraphe `multi`. La coupe obtenue sera une coupe minimale avec une probabilité supérieure à `certitude`.

Remarque : si le multigraphe en entrée contient plus de 3 composantes connexes, l'algorithme `karger_certitude`, qui se base sur la fonction `karger` précédente, retournera un multigraphe qui contient autant de sommets que de composantes connexes et aucune arête. La taille de la coupe minimale sera 0.